Họ và tên: Nguyễn Hữu Hạo

MSSV: 2474802010108

BÁO CÁO THỰC HÀNH LAB 5

OPTIMIZER TRONG DEEP LEARNING VÀ REGULARIZATION

Báo cáo thực hiện toàn bộ quy trình dự báo churn bằng PyTorch, ba thí nghiệm chính và các Bài 1 đến 8. Trong bốn ứng viên triển khai chuẩn, Adam với lr = 0,001, weight_decay = 0,01 và batch size 64 được chọn bằng validation; test accuracy đạt 84,50%.

1 Dữ liệu và thiết lập thực nghiệm

Bảng 1 Tám thuộc tính đầu vào của bài toán churn

Thuộc tính	Ý nghĩa	Khoảng mô phỏng
tenure_months	Số tháng sử dụng dịch vụ	1–72
monthly_spend	Chi tiêu mỗi tháng	20–130
support_tickets	Số lần liên hệ hỗ trợ	0–8
usage_hours	Mức sử dụng dịch vụ	2–90
late_payments	Số lần thanh toán trễ	0–5
satisfaction	Điểm hài lòng	1–5
contract_months	Thời hạn hợp đồng	1, 12 hoặc 24
discount_rate	Tỷ lệ ưu đãi	0–0,30

Dữ liệu và cấu hình thực nghiệm

```
So khach hang: 2000 | So dac trung: 8
Class balance: [1000, 1000]
Train: 1200 (600 / 600)
Validation: 400 (200 / 200)
Test: 400 (200 / 200)
MLP: 8 -> 32 -> 16 -> 1 | Tham so: 833
Seed: 42 | CPU threads: 1
```

Hình 1 Kết quả tạo dữ liệu và chia tập

Nhận xét: Dữ liệu là mô phỏng theo đúng hàm create_churn_data trong đề, không phải khách hàng thật. Nhãn churn = 1 nếu score vượt median, tạo hai lớp cân bằng. StandardScaler chỉ fit trên 1.200 mẫu train; validation và test chỉ được transform để tránh rò rỉ dữ liệu.

Môi trường chạy: Python 3.12.14, PyTorch 2.14.1+cpu, NumPy 2.3.5, scikit-learn 1.8.0. Cố định seed 42, CPU một luồng. Runtime là thời gian vòng huấn luyện và đánh giá mỗi epoch, không bao gồm khởi tạo dữ liệu và optimizer.

2 Phát biểu bài toán tối ưu và Bài 1

$$\theta^* = argmin_\theta J(\theta)$$

$$J(\theta) = BCE_{train}(\theta) + \lambda \|\theta\|^2$$

θ gồm weights và biases của ba lớp Linear. BCE là lỗi phân loại nhị phân trung bình trên train; dự đoán tự tin nhưng sai bị phạt mạnh. λ điều chỉnh mức phạt L2; ‖θ‖² là tổng bình phương các tham số. θ* là bộ trọng số học được, không bảo đảm tối ưu toàn cục.

Bảng 2 Biến quyết định objective hyperparameters và constraints

Thành phần	Phát biểu trong bài
Decision variables	Weights và biases của MLP 8 → 32 → 16 → 1.
Objective ở cấp tham số	Giảm BCE trên train, có penalty nếu regularize.
Hyperparameters	Optimizer, learning rate, batch size, weight_decay và số epoch.
Objective chọn cấu hình	Maximize validation accuracy; hòa thì ưu tiên validation loss thấp hơn.
Constraints chuẩn	Validation accuracy ≥ 84%; |Train acc − Val acc| ≤ 0,05; epoch ≤ 100.

Bảng 3 Phân loại parameter và hyperparameter

Đại lượng	Phân loại	Cách được xác định
weights, biases	Model parameters	Optimizer cập nhật từ gradient.
learning rate	Hyperparameter	Chọn trước lần train.
batch size	Hyperparameter	Chọn số mẫu cho mỗi bước cập nhật.
weight_decay	Hyperparameter	Chọn mức regularization.
số epoch	Hyperparameter và budget	Giới hạn số lượt duyệt train.

$$(8 \times 32 + 32) + (32 \times 16 + 16) + (16 \times 1 + 1) = 833$$

Code chính của bước tối ưu

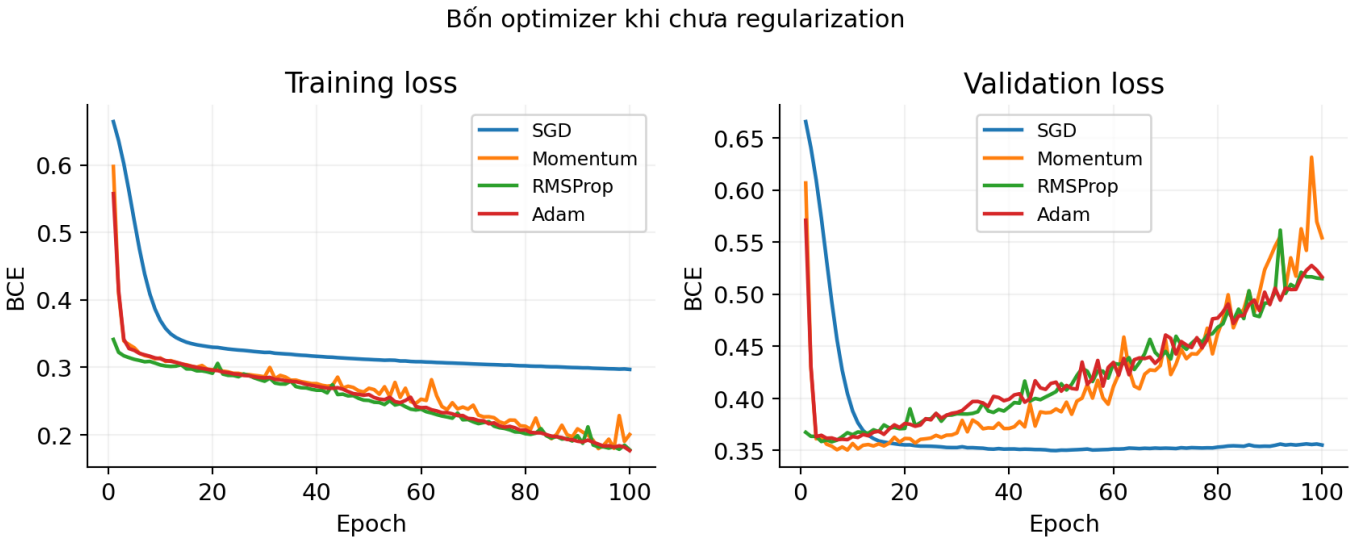
```
optimizer.zero_grad()
logits = model(xb)
loss = nn.BCEWithLogitsLoss()(logits, yb)
loss.backward()
optimizer.step()
```

Nhận xét: backward tính gradient, còn step cập nhật θ. Một cấu hình khả thi phải thỏa đồng thời ba ràng buộc trên validation. Mô hình triển khai chỉ cần scaler và ChurnMLP mang θ*; optimizer không cần cho forward khi dự đoán, nhưng cần nếu tiếp tục huấn luyện.

Trong thí nghiệm dùng Adam mặc định với coupled weight decay, không dùng AdamW. Hệ số weight_decay = w tạo thêm gradient wθ, tương ứng penalty (w/2)‖θ‖². Các cột Train loss và Val loss bên dưới chỉ báo BCE, không cộng penalty.

3 Thí nghiệm so sánh bốn optimizer

Cùng dữ liệu, seed 42, MLP, B = 64, 100 epoch và weight_decay = 0. Learning rate khởi đầu lần lượt là 0,05; 0,03; 0,003; 0,003 theo đề.



Hình 2 Training loss và validation loss khi chưa regularization

Bảng 4 Kết quả cuối epoch 100 của thí nghiệm 1

Optimizer	lr	Train loss	Val loss	Train acc	Val acc	Gap	Epoch ≥84%	Runtime s
SGD	0.0500	0.2970	0.3553	86.50%	85.75%	0.0075	11	1.09
Momentum	0.0300	0.2006	0.5542	90.50%	80.75%	0.0975	3	1.17
RMSProp	0.0030	0.1780	0.5150	93.58%	82.25%	0.1133	1	1.34
Adam	0.0030	0.1768	0.5166	93.00%	83.00%	0.1000	3	1.51

Nhận xét: Adam đạt training BCE thấp 0,1768 nhưng validation BCE là 0,5166 và gap = 0,1000. SGD có training BCE cao hơn 0,2970 nhưng validation BCE thấp hơn 0,3553, accuracy 85,75% và gap chỉ 0,0075. Momentum, RMSProp và Adam có dấu hiệu overfit trong cấu hình chưa regularize; loss train thấp không đủ để chọn mô hình triển khai.

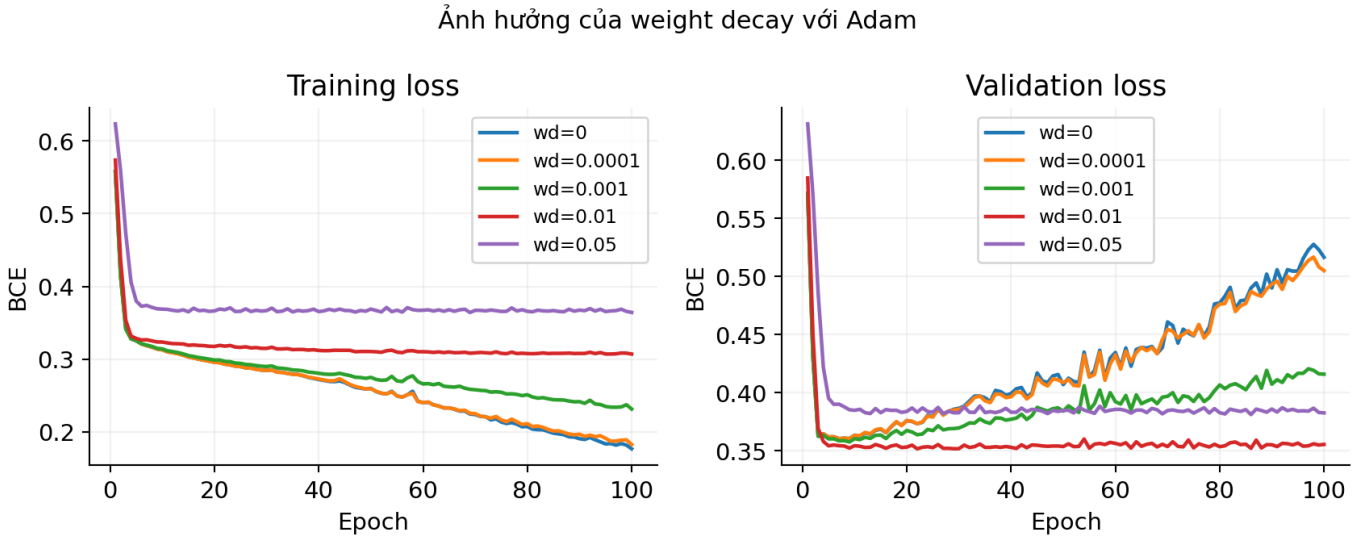
Epoch ≥84% là epoch đầu tiên có ít nhất 336/400 dự đoán validation đúng. Accuracy được tính từ số mẫu đúng để 336/400 = 84% vẫn được tính đạt ngưỡng, tránh sai số float32 ở đúng biên. Runtime đo một lần trên cùng CPU, không là kết luận về tốc độ trên mọi phần cứng.

Cấu hình optimizer

```
optim.SGD(model.parameters(), lr=0.05)
optim.SGD(model.parameters(), lr=0.03, momentum=0.9)
optim.RMSprop(model.parameters(), lr=0.003, alpha=0.99)
optim.Adam(model.parameters(), lr=0.003)
```

4 Thí nghiệm ảnh hưởng của L2 regularization

Giữ Adam, lr = 0,003, B = 64 và 100 epoch. Chỉ đổi weight_decay trong {0; 0,0001; 0,001; 0,01; 0,05} để tách ảnh hưởng của mức phạt.



Hình 3 Ảnh hưởng của weight decay lên train loss và validation loss

Bảng 5 Kết quả cuối epoch 100 khi thay mức L2

weight decay	Train loss	Val loss	Train acc	Val acc	Gap	Runtime s	Khả thi
0.0000	0.1768	0.5166	93.00%	83.00%	0.1000	1.51	Không
0.0001	0.1827	0.5050	92.25%	84.75%	0.0750	1.48	Không
0.0010	0.2314	0.4159	90.08%	84.75%	0.0533	1.54	Không
0.0100	0.3070	0.3554	85.67%	85.25%	0.0042	1.64	Có
0.0500	0.3642	0.3826	83.92%	85.50%	-0.0158	1.58	Có

Nhận xét: Tăng weight_decay từ 0 lên 0,01 làm train BCE tăng từ 0,1768 lên 0,3070, nhưng validation BCE giảm từ 0,5166 xuống 0,3554 và gap giảm từ 0,1000 xuống 0,0042. Đây là đánh đổi có lợi giữa khớp train và tổng quát hóa.

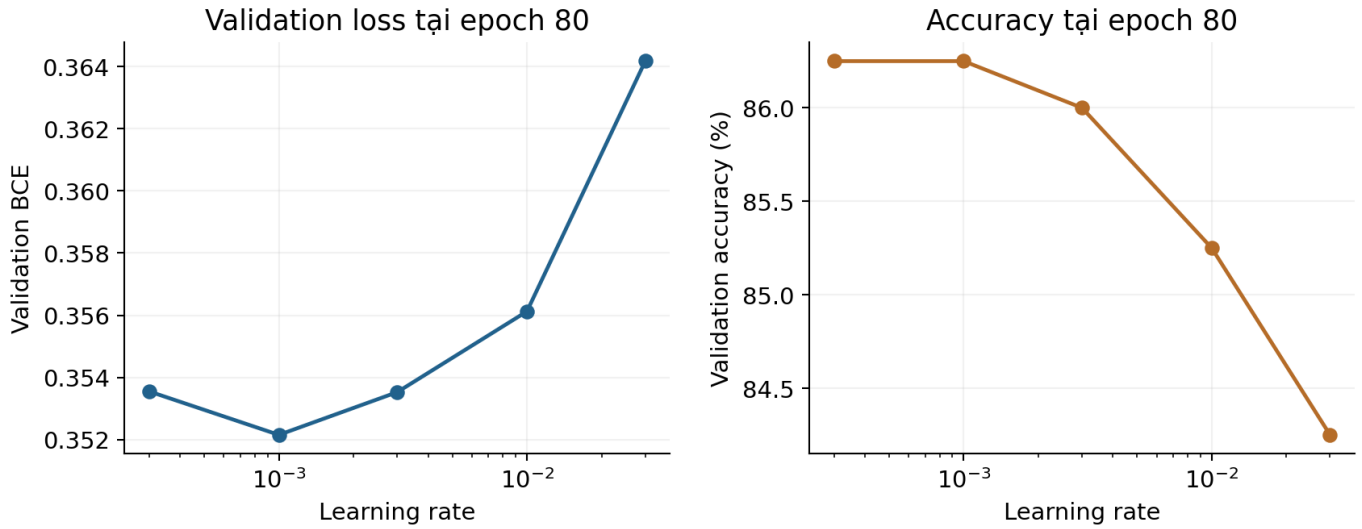
Mức 0,05 vẫn khả thi nhưng validation BCE tăng lại 0,3826. Vì vậy giữ weight_decay = 0,01 cho bước khảo sát learning rate tiếp theo theo quy trình của đề. Đây là lựa chọn cân bằng BCE, gap và accuracy, không khẳng định 0,01 tốt cho mọi dữ liệu.

Code chính

```
for wd in [0.0, 1e-4, 1e-3, 1e-2, 5e-2]:
    run = train_model('Adam', lr=0.003,
                      weight_decay=wd, epochs=100)
```

5 Thí nghiệm độ nhạy của learning rate

Giữ Adam, weight_decay = 0,01, B = 64. Mỗi lr chạy đúng 80 epoch; không so kết quả epoch 80 với epoch 100 như cùng một budget.



Hình 4 Độ nhạy validation loss và accuracy theo learning rate

Bảng 6 Kết quả learning rate tại epoch 80

lr	Train loss	Val loss	Train acc	Val acc	Gap	Epoch ≥84%
0.0003	0.3236	0.3536	85.00%	86.25%	-0.0125	19
0.0010	0.3152	0.3522	85.42%	86.25%	-0.0083	5
0.0030	0.3076	0.3535	86.08%	86.00%	0.0008	3
0.0100	0.3104	0.3561	86.08%	85.25%	0.0083	1
0.0300	0.3276	0.3642	84.42%	84.25%	0.0017	2

Nhận xét: lr = 0,0003 và 0,001 cùng đạt validation accuracy 86,25%. lr = 0,001 có validation BCE thấp hơn, 0,3522 so với 0,3536, nên được chọn theo quy tắc phá hòa. lr = 0,03 có loss 0,3642 và accuracy 84,25%, kém hơn trong cùng budget.

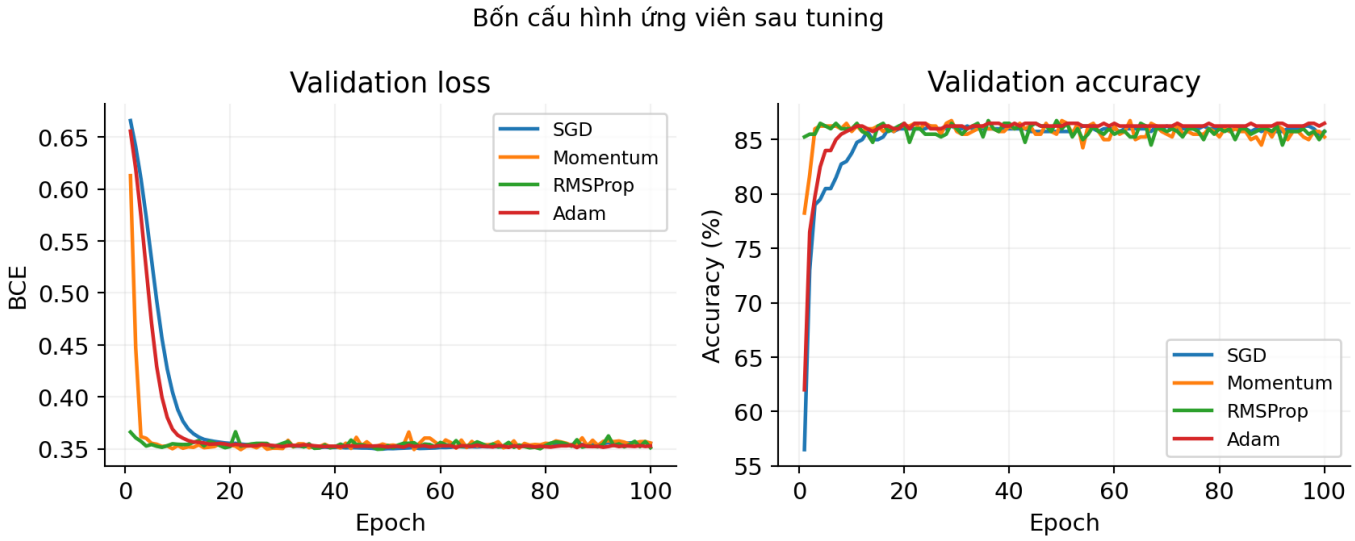
Cấu hình Adam đưa vào vòng ứng viên chuẩn là lr = 0,001, weight_decay = 0,01, B = 64. Sau đó huấn luyện 100 epoch để so với ba optimizer còn lại theo cùng budget.

Code lựa chọn từ validation

```
best = min(learning_rates,
        key=lambda run: (-run['row']['val_acc'], run['row']['val_loss']))
# Chỉ dùng validation, chưa đánh giá test.
```

6 Nghiệm khả thi và cấu hình triển khai chuẩn

Tập ứng viên triển khai chuẩn được giới hạn trước ở bốn cấu hình dưới đây theo quy trình Mục 3.11. Các bài batch, L1, early stopping và budget 30 là thí nghiệm riêng; không dùng kết quả test để thay đổi quyết định này.



Hình 5 Validation của bốn ứng viên sau tuning

Bảng 7 Kiểm tra accuracy gap và budget 100 epoch

Optimizer	lr	wd	Val loss	Val acc	Gap	Epoch ≥84%	Khả thi
SGD	0.0500	0.00	0.3553	85.75%	0.0075	11	Có
Momentum	0.0300	0.01	0.3558	85.25%	0.0050	3	Có
RMSProp	0.0030	0.01	0.3513	85.75%	0.0058	1	Có
Adam	0.0010	0.01	0.3529	86.50%	-0.0133	5	Có

$$h^* = (Adam, \eta = 0.001, B = 64, weight_{decay} = 0.01)$$

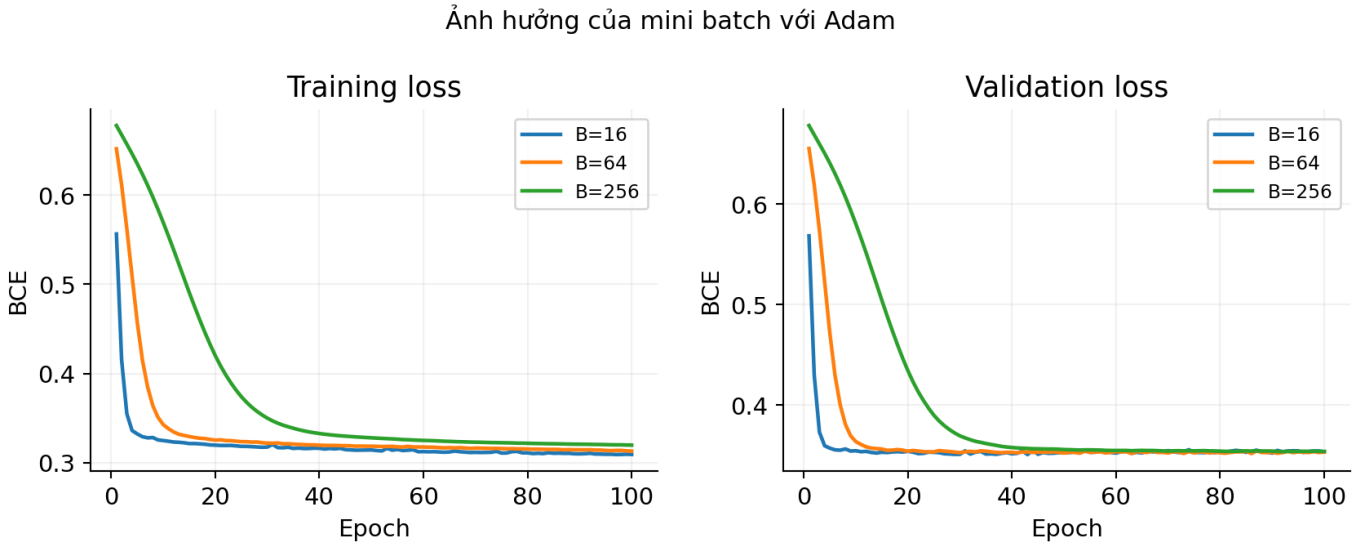
Nhận xét: Cả bốn ứng viên thỏa validation accuracy ≥ 84%, |gap| ≤ 0,05 và epoch ≤ 100. Adam có validation accuracy cao nhất 86,50%, gap = −0,0133 và validation BCE = 0,3529. RMSProp có BCE nhỏ hơn 0,3513 nhưng accuracy 85,75%, nên không thắng theo objective ưu tiên accuracy đã đặt trước.

Gap âm nhỏ nghĩa là accuracy trên tập validation đang cao hơn train; điều này có thể xảy ra do cách chia mẫu và không tự thể hiện lỗi tính toán. |gap| được dùng để kiểm tra ràng buộc. Cấu hình được chọn là tốt nhất trong tập bốn ứng viên, không phải nghiệm toàn cục của mọi tổ hợp hyperparameter.

7 Bài 2 Ảnh hưởng của mini batch

$$Steps\ mỗi\ epoch\ =\ ceil(1200\ /\ B)$$

Giữ Adam, lr = 0,001, weight_decay = 0,01 và 100 epoch. Vì batch cuối vẫn được dùng, số bước là ceil thay vì chỉ lấy phần nguyên.



Hình 6 Train loss và validation loss khi thay batch size

Bảng 8 Số bước cập nhật runtime và chất lượng của ba batch

B	Bước mỗi epoch	Tổng bước 100 epoch	Val loss	Val acc	Runtime s	Dao động Val loss
16	75	7500	0.3532	86.00%	4.84	0.000924
64	19	1900	0.3529	86.50%	1.52	0.000841
256	5	500	0.3535	86.25%	0.64	0.000147

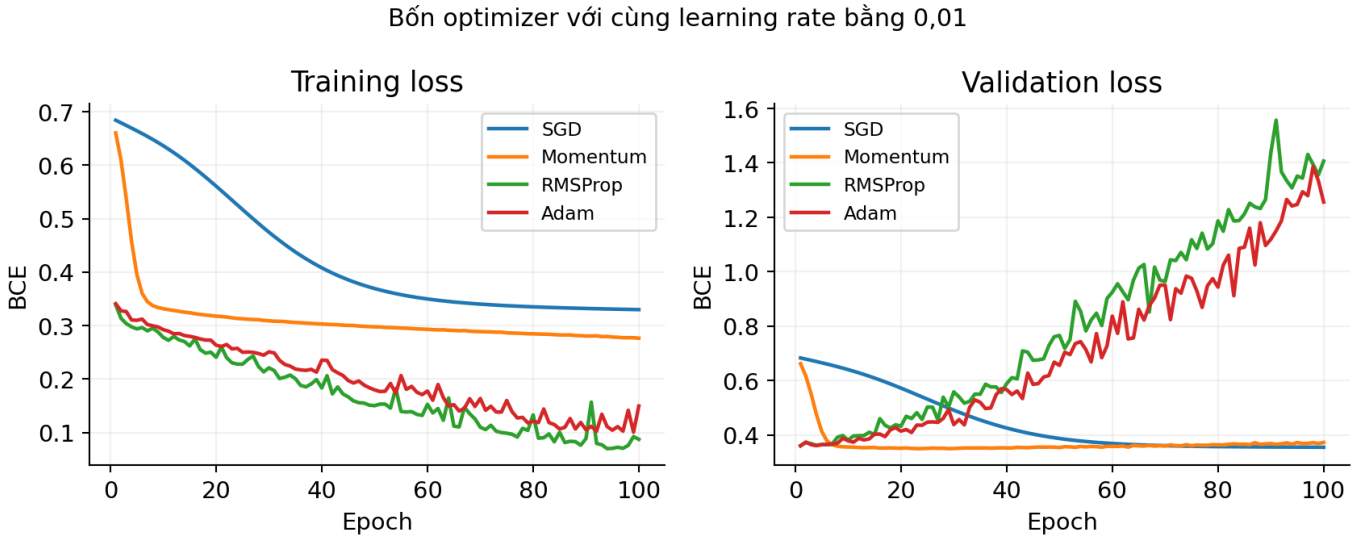
Nhận xét: B = 16, 64, 256 tạo lần lượt 7.500, 1.900, 500 bước cập nhật. Runtime đo được là 4.84; 1.52; 0.64 giây. B = 256 nhanh nhất và validation loss ít dao động nhất trong lần chạy này; B = 64 có accuracy cao nhất 86,50%.

Dao động là trung bình |ValLoss(t) − ValLoss(t−1)| trong 20 epoch cuối; đây chỉ là chỉ số mô tả của thí nghiệm. Cùng số epoch không đồng nghĩa cùng số lần cập nhật.

Nếu hạn chế thời gian CPU, chọn B = 256 cho mạng nhỏ này vì accuracy 86,25% vẫn gần B = 64 nhưng chạy nhanh hơn. Nếu ưu tiên cân bằng bộ nhớ và accuracy, B = 64 là phương án chính. Batch lớn cần nhiều bộ nhớ activation hơn; thí nghiệm chưa đo RAM nên không suy ra batch nào phù hợp với mọi máy ít bộ nhớ.

8 Bài 3 So sánh khi dùng cùng learning rate

Cả bốn optimizer dùng lr = 0,01, B = 64, 100 epoch và weight_decay = 0 để so trực tiếp với thí nghiệm chưa regularization.



Hình 7 Bốn optimizer cùng learning rate nhưng quỹ đạo tối ưu khác nhau

Bảng 9 Kết quả cùng lr bằng 0,01 và so sánh với Mục 3

Optimizer	Val loss	Val acc	Gap	Epoch ≥84%	Δ Val acc so với TN1
SGD	0.3556	86.00%	-0.0175	50	0.25 điểm %
Momentum	0.3733	85.50%	0.0208	6	4.75 điểm %
RMSProp	1.4065	82.00%	0.1408	2	-0.25 điểm %
Adam	1.2559	79.00%	0.1383	1	-4.00 điểm %

Nhận xét: SGD đạt 86,00%; Momentum 85,50%; RMSProp 82,00%; Adam 79,00%. RMSProp và Adam có validation BCE tăng mạnh trong khi training BCE giảm. Cùng một lr chỉ cố định giá trị số, vì Momentum tích lũy hướng còn RMSProp/Adam điều chỉnh bước theo lịch sử gradient; bước cập nhật hiệu dụng vẫn khác nhau.

Đánh giá công bằng hơn cần tune lr riêng cho từng optimizer trong cùng ngân sách tìm kiếm, cùng kiến trúc, dữ liệu và quy tắc chọn validation. Không kết luận optimizer nào luôn tốt nhất chỉ từ một lr dùng chung.

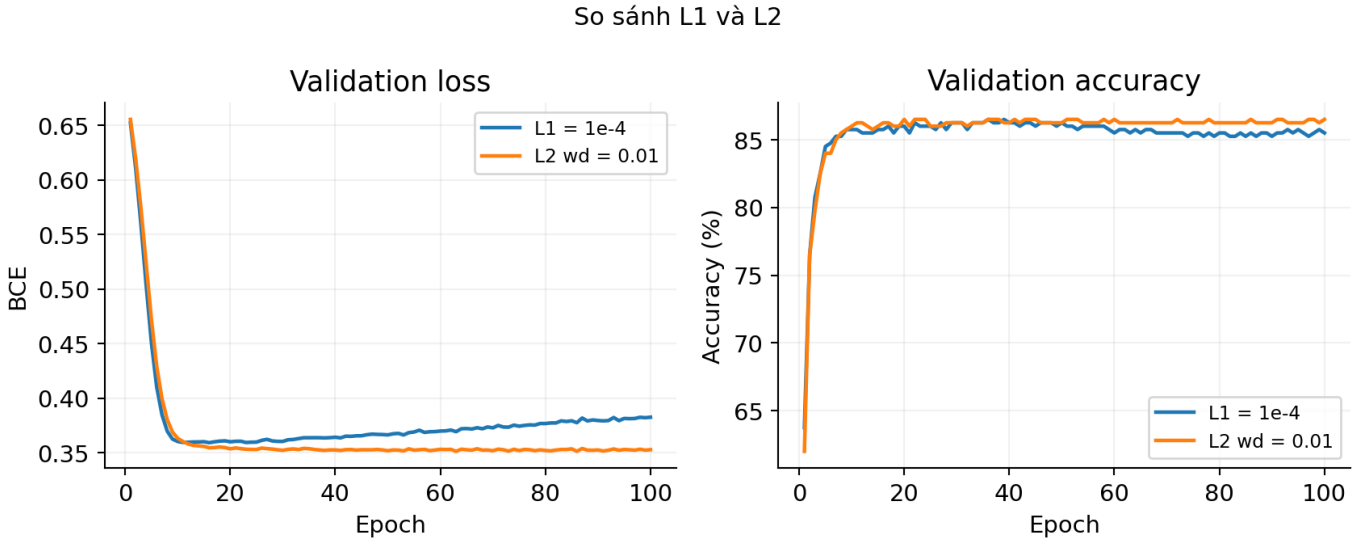
9 Bài 4 L1 regularization và nghiệm sparse

$$J(\theta) = BCE_{train}(\theta) + \lambda_1 \Sigma |\theta_i|$$

Giữ Adam, lr = 0,001, B = 64, 100 epoch; với L1 đặt weight_decay = 0 và λ_1 = 0,0001. Penalty cộng trên toàn bộ parameters theo code đề. Sparsity bên dưới chỉ đếm 784 weights, không đếm 49 biases.

Code chính

```
l1_penalty = sum(p.abs().sum() for p in model.parameters())
loss_total = loss_data + 1e-4 * l1_penalty
loss_total.backward()
optimizer.step()
```



Hình 8 Validation loss và accuracy của L1 và L2

Bảng 10 Accuracy và tỷ lệ weights gần bằng 0

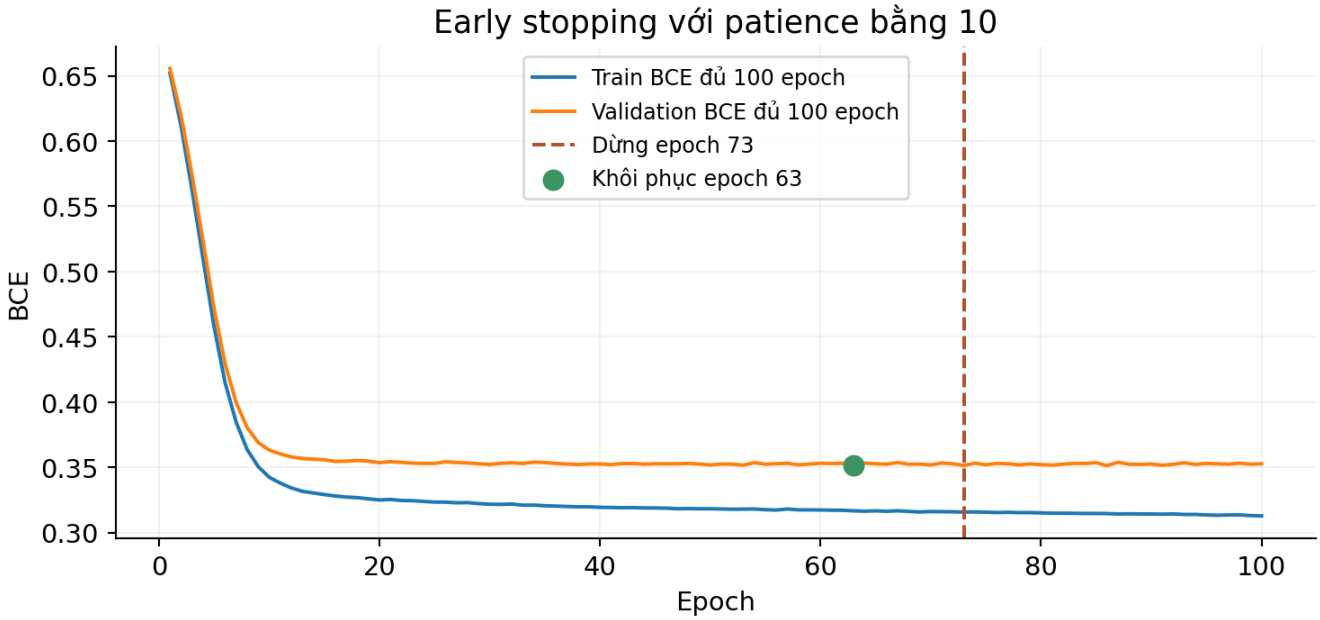
Cấu hình	Val loss	Val acc	Weights |w| < 0,001	Tỷ lệ gần 0
L1 λ_1 = 1e-4	0.3826	85.50%	111/784	14.16%
L2 wd = 0,01	0.3529	86.50%	134/784	17.09%

Nhận xét: L1 đạt 85,50% accuracy và 111/784 weights gần 0, tương ứng 14,16%; L2 đã chọn đạt 86,50% và 134/784, tương ứng 17,09%. Ở các mức phạt cụ thể này, L1 chưa tạo tỷ lệ gần 0 cao hơn L2. Cường độ phạt khác nhau nên không dùng kết quả này để phủ nhận hoặc khẳng định L1 luôn sparse hơn.

L1 có thể đẩy nhiều weights về gần 0, giúp xác định kết nối ít quan trọng để pruning. Weights gần 0 chưa bị xóa: MLP hiện vẫn có 833 tham số. Muốn giảm kích thước triển khai cần pruning hoặc cơ chế rút gọn rồi kiểm tra lại chất lượng.

10 Bài 5 Early stopping theo validation loss

Dùng Adam lr = 0,001, weight_decay = 0,01, B = 64, tối đa 100 epoch. Dừng khi 10 epoch liên tiếp không giảm validation loss; min_delta = 0. Lưu và khôi phục state_dict tại epoch tốt nhất.



Hình 9 Epoch dừng và epoch được khôi phục với patience bằng 10

Bảng 11 So sánh dừng sớm với train đủ budget

Cách train	Epoch đã chạy	Epoch trọng số	Val loss	Val acc	Runtime s
Đủ 100 epoch	100	100	0.3529	86.50%	1.52
Early stopping	73	63	0.3514	86.25%	1.13

Code chính

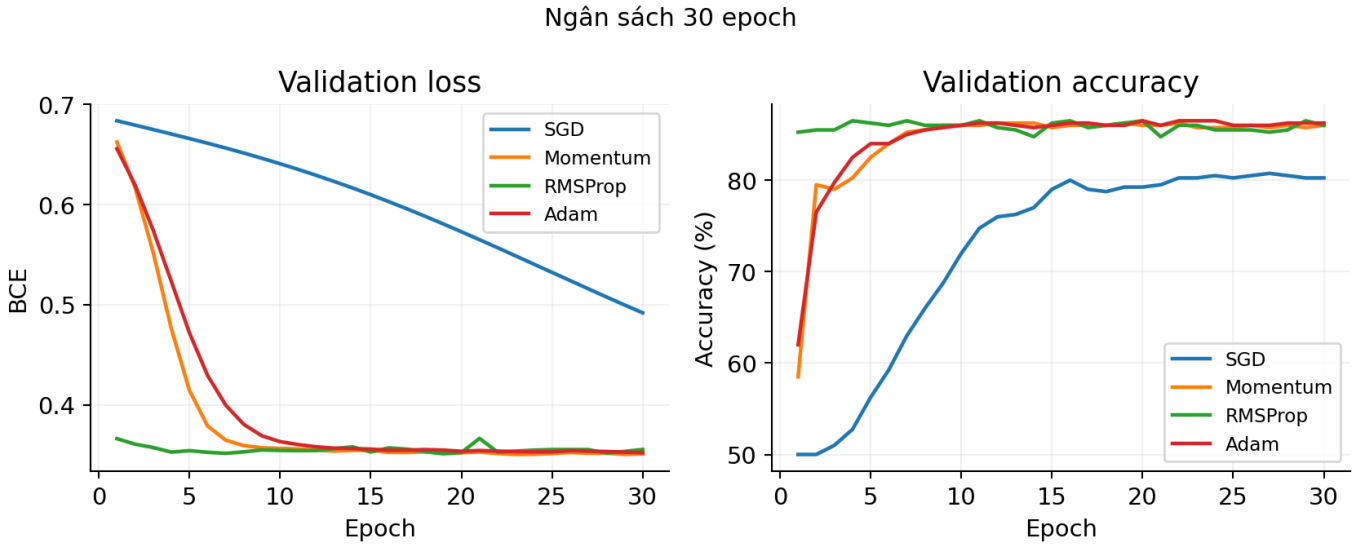
```
if val_loss < best_loss:
    best_loss = val_loss
    best_state = copy.deepcopy(model.state_dict())
    wait = 0
else:
    wait += 1
if wait >= 10:
    break
model.load_state_dict(best_state)
```

Nhận xét: Quá trình dừng ở epoch 73 và khôi phục trọng số epoch 63. Validation BCE tốt nhất là 0,3514, thấp hơn 0,3529 khi dùng trọng số epoch 100, dù accuracy thấp hơn nhẹ 86,25% so với 86,50%. Runtime giảm từ khoảng 1,52 xuống 1,13 giây trong lần đo.

Early stopping giới hạn thời gian mô hình tiếp tục khớp train sau giai đoạn validation còn cải thiện, nên có tác dụng regularization theo thời gian. Cấu hình này là thí nghiệm bổ sung, không dùng test để quyết định epoch dừng.

11 Bài 6 Budget huấn luyện chỉ còn 30 epoch

Với mỗi optimizer, chọn cấu hình tốt nhất từ các lần chạy đủ 100 epoch đã thử (không gồm L1 hoặc dừng sớm), rồi train lại đúng 30 epoch từ cùng seed. Objective là maximize validation accuracy với epoch ≤ 30 và |gap| ≤ 0,05.



Hình 10 Validation trong ngân sách 30 epoch

Bảng 12 Nghiệm dưới budget 30 epoch

Optimizer	lr	wd	B	Val loss	Val acc	Gap	Đạt |gap|≤0,05
SGD	0.0100	0.00	64	0.4918	80.25%	0.0242	Có
Momentum	0.0100	0.00	64	0.3510	86.00%	0.0000	Có
RMSProp	0.0030	0.01	64	0.3554	86.00%	-0.0067	Có
Adam	0.0010	0.01	64	0.3523	86.25%	-0.0125	Có

Nhận xét: Adam vẫn được chọn, với lr = 0,001, wd = 0,01 và B = 64; validation accuracy 86,25%, gap = −0,0125, validation BCE = 0,3523. Optimizer và hyperparameters giống cấu hình chuẩn, nhưng θ và budget khác nên accuracy giảm 0,25 điểm phần trăm so với 100 epoch.

SGD lr = 0,01 đạt 86,00% ở 100 epoch nhưng chỉ 80,25% ở epoch 30. Cấu hình này vẫn thỏa hai ràng buộc riêng của Bài 6, nhưng chưa đạt mốc 84% của bài toán triển khai chuẩn. Một optimizer tốt khi được train lâu có thể kém phù hợp nếu budget ngắn; cần đánh giá lại trong đúng ngân sách.

12 Bài 7 Ràng buộc chặt và đánh giá test cuối

Thay yêu cầu thành validation accuracy ≥ 87% và |gap| ≤ 0,03. Kiểm tra toàn bộ 28 cấu hình độc lập đã chạy trong các thí nghiệm, kể cả L1, early stopping và budget 30.

Bảng 13 Kiểm tra ràng buộc chặt hơn cho bốn ứng viên chuẩn

Ứng viên chuẩn	Val acc	|Gap|	Đạt 87%	Đạt gap	Khả thi chặt
SGD	85.75%	0.0075	Không	Có	Không
Momentum	85.25%	0.0050	Không	Có	Không
RMSProp	85.75%	0.0058	Không	Có	Không
Adam	86.50%	0.0133	Không	Có	Không

Nhận xét: Accuracy cao nhất trong 28 cấu hình là 86,50%, nên chưa tìm thấy nghiệm khả thi đạt đồng thời 87% và |gap| ≤ 0,03 trong tập ứng viên và ngân sách hiện tại. Điều này không chứng minh bài toán không thể có nghiệm trong mọi cấu hình khác.

Hướng mở rộng: (1) tune thêm learning rate và weight decay trong ngân sách công bằng; (2) thử batch size và early stopping khác; (3) cải thiện đặc trưng hoặc thu thập dữ liệu ít nhiễu hơn; (4) xem xét kiến trúc khác với một tập validation phù hợp. Không âm thầm hạ ngưỡng hoặc tuning theo test để làm kết quả đẹp hơn.

Test độc lập sau khi chốt cấu hình chuẩn

Kết quả đánh giá test sau khi chốt cấu hình

```
Chosen: Adam, lr=0.001, wd=0.01
B=64, epochs=100
Test BCE: 0.30175069
Test accuracy: 0.8450
Confusion matrix: [[174, 26], [36, 164]]
Test forward calls: 1
```

Hình 11 Kết quả test của mô hình Adam đã chọn từ validation

Bảng 14 Các chỉ số lấy từ cùng một lần dự đoán test

Test BCE	Accuracy	Precision churn	Recall churn	F1 churn
0.3018	84.50%	86.32%	82.00%	84.10%

Ma trận nhầm lẫn theo hàng nhãn thật 0/1, cột nhãn dự đoán 0/1 là [[174, 26], [36, 164]]. Test chỉ được forward một lần sau khi đã hoàn thành lựa chọn bằng validation; các metric trên dùng cùng kết quả dự đoán. Không huấn luyện hoặc đổi hyperparameter sau khi xem test. Số liệu là dữ liệu mô phỏng, chưa phản ánh hiệu năng trên khách hàng thực.

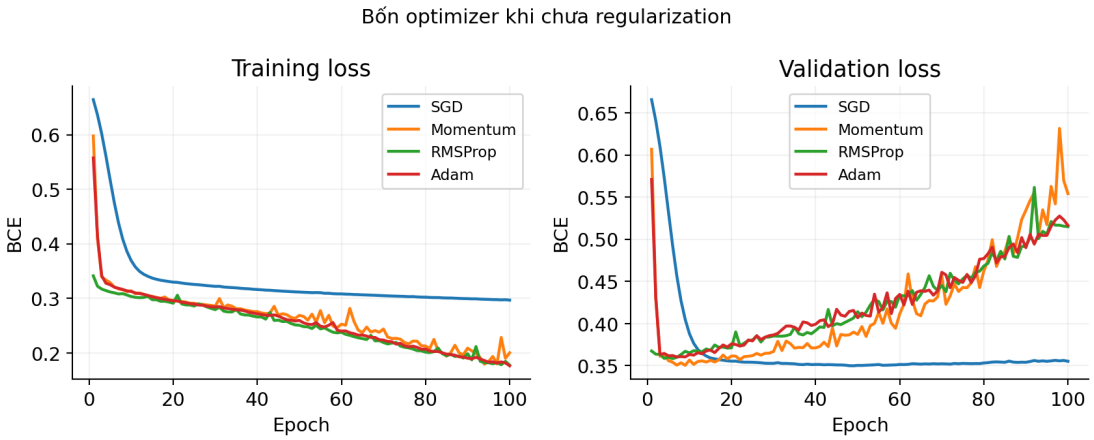
13 Bài 8 Báo cáo quyết định trong một trang

Dữ liệu: 2.000 khách hàng mô phỏng, 8 thuộc tính, nhãn churn 0/1 cân bằng. Chia train/validation/test = 1.200/400/400; scaler chỉ fit train. Biến θ gồm 833 weights và biases của ChurnMLP 8 → 32 → 16 → 1.

$$J(\theta) = BCE_{train}(\theta) + \lambda \|\theta\|^2$$

Bảng 15 Phạm vi tìm kiếm thực nghiệm

Nhóm	Các giá trị đã thử
Optimizer	SGD; Momentum 0,9; RMSProp α=0,99; Adam.
Learning rate	0,0003; 0,001; 0,003; 0,01; 0,03; SGD 0,05.
Regularization và batch	wd = 0; 1e-4; 1e-3; 0,01; 0,05; L1 = 1e-4; B = 16; 64; 256.
Budget	80/100 epoch ở TN chính; 30 epoch và early stopping ở bài bổ sung.



Hình 12 Hai convergence curve train BCE và validation BCE của bốn optimizer

Bảng 16 Ràng buộc accuracy ≥84% gap tuyệt đối ≤0,05 epoch ≤100

Ứng viên chuẩn	Val loss	Val acc	Gap	Khả thi
SGD	0.3553	85.75%	0.0075	Có
Momentum	0.3558	85.25%	0.0050	Có
RMSProp	0.3513	85.75%	0.0058	Có
Adam	0.3529	86.50%	-0.0133	Có

Quyết định: Chọn Adam lr = 0,001, B = 64, wd = 0,01 và 100 epoch trong bốn ứng viên chuẩn vì validation accuracy cao nhất 86,50%; validation BCE 0,3529, gap −0,0133. Không chọn theo training loss thấp nhất.

Test cuối: BCE 0,3018; accuracy 84,50% từ một lần dự đoán 400 mẫu. Không dùng test để chọn cấu hình, epoch hoặc fit scaler. Ràng buộc 87% và |gap|≤0,03 chưa có nghiệm trong 28 cấu hình đã thử.

Triển khai scaler đã fit và ChurnMLP với trọng số θ* của cấu hình trên. Adam đã dùng gradient để tìm θ*; khi suy luận chỉ cần scaler và forward của mạng. Kết luận giới hạn ở dữ liệu mô phỏng, một cách chia mẫu và seed 42.